

Advanced Data Management

Incomplete Information

Domenico Fabio Savo

*Corso di laurea magistrale
INGEGNERIA INFORMATICA
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione
University of Bergamo*

Outline of the course

1. Introduction to propositional logic
 2. Introduction to First-order Logic
 3. Relational Calculus
 4. Information Integration **S**ystems (IIS)
 5. Logical formalization of IISs
 6. Mapping between **Global Schema** e **Data Sources**
 7. **Incomplete information databases**
 8. Query answering over IISs
 9. **O**ntology-**B**ased **D**ata **M**anagement (OBDM)
 10. Description Logic Ontologies
 11. Query answering in ontologies
 12. Query answering in OBDM
-

Query Answering Over Databases (recap)

Databases as an Interpretation

We can see a database instance **D** as a (finite) interpretation I .

- Our **domain** is the set **dom(D)** of attribute values
- Our **alphabet** is formed by:
 - the set **Rel**s of relational predicates R/n
 - the set **Cons** of constants (we have a function c with arity 0 (constant) for each element c in **dom**).

We use **databases as interpretations**. Hence, given a database **D** over a schema $S = \{R_1, \dots, R_n\}$, we implicitly see **D** as the interpretation $I = (\mathbf{dom(D)}, \cdot^I)$, where:

- $R^I = D(R)$ for each $R \in S$ and $R^I = \emptyset$ for each $R \notin S$
- $c^I = c$ for each $c \in \mathbf{Cons}$ (each constant is interpreted into itself)

Answers for Complete Database

Consider to have instance **D** of a schema **S** over a domain *dom*, we can see **D** as a FOL **interpretation**.

So, given a FOL formula φ and an assignment α , we know if $\mathbf{D}, \alpha \models \varphi$

The answers for a query $q = \{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$ over a database **D** are the set $q(\mathbf{D})$ defined as follows:

$$q(\mathbf{D}) = \{ \langle c_1, \dots, c_n \rangle \in D^n \mid \mathbf{D}, \langle c_1, \dots, c_n \rangle \models \varphi(x_1, \dots, x_n) \}$$

Example

Consider the following database **D** and the FOL query:

$$q = \{(x) \mid \exists y. \exists z. (Customers(y, x) \wedge Orders(z, y, P002))\}$$

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Customers

Code	Name
C001	Eryn
C002	Dylan
C003	Cora

We have that:

$$q(D) = \langle \text{Dylan} \rangle$$

Orders

Code	Customer	Product
O001	C001	P001
O002	C002	P002
O003	C001	P001

Incomplete Data

More than Completeness

The general notion of sound mappings gives rise to **incomplete information**

Consider for instance the following (G)LAV mapping assertion:

$$m: \forall t, r. r_2(t, r) \rightarrow \exists y, d. \text{Review}(t, r) \wedge \text{Movie}(t, y, d)$$

Given a source database **D**, in general there are several solutions for a set of (G)LAV assertions (i.e., different databases that are legal w.r.t. **G** and that satisfy **M** w.r.t. **D**)

→ **Mapping may generate incompleteness**

Especially for *materialization-based techniques*, we need a mechanism to capture incompleteness in databases.

So far, we considered databases with **complete information**: a tuple is either in a relation or it is not

We need a new notion of databases that allows incompleteness

Complete Databases (1/2)

So far we always assumed that databases are complete.

In what sense?

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Customers

Code	Name
C001	Eryn
C002	Dylan
C003	Cora

Orders

Code	Customer	Product
O001	C001	P001
O002	C002	P002
O003	C001	P001

Complete Databases (2/2)

We know **everything** about the objects represented in the database.

- We know the color of all products, the names of all customers...

We know **ALL** the objects of interest

- We know all products, all customers...

We also know what an object is **NOT**

- For example, we know that P001 is NOT a customer code.
-

Queries Over Complete Databases

What are the answers for the following queries?

- Return code and color of all the products.
 - $\{(x, y) \mid \text{Products}(x, y)\}$
 - **Answers:** $\{ \langle \text{P001}, \text{Red} \rangle, \langle \text{P002}, \text{Green} \rangle, \langle \text{P003}, \text{Blue} \rangle \}$
- Return the codes of products that are not Red.
 - $\{(x) \mid \exists y. \neg \text{Products}(x, \text{Red}) \wedge \text{Products}(x, y)\}$
 - **Answers:** $\{ \langle \text{P002} \rangle, \langle \text{P003} \rangle \}$

Products

Code	Color
P001	Red
P002	Green
P003	Blue

Incomplete Data

Data may be incomplete in two different ways:

- Some **value** may be missing.
 - E.g., we may **not** know the color of P003
 - Return code and color of all the products.
 - $\{(x, y) \mid Products(x, y)\}$
 - **Answers:** $\{(P001, Red), (P002, Green), (P003, ??)\}$
- Some **tuple** may be missing
 - E.g., there may be more products than those in the table Products.
 - Return the codes of products that are not Red.
 - $\{(x) \mid \forall y. \neg Products(x, Red)\}$
 - **Answers:** $\{P002, P003, \dots\}$ what else?

Representing Incomplete Data

This scenario poses us two main challenges:

1. **How do we represent the missing information?**
 2. **What does it mean to answer a query over such database?**
-

Incomplete Information in the Relational Model

Incomplete Databases: Real-World Solutions

Often “normal” values of the domain (such as 0, the empty string, “9999”, etc) are used to represent **missing information**.

However, this is a **mistake** for a host of reasons:

- There might not be any “unused values”.
- Values that are “unused” up to a certain point, might become used later.
- The token used for the missing value carries no meaning. Different programmers\users may use different tokens.

How do we model (correctly) incomplete information in the relational model?

Incomplete Information in the Relational Model

In the relational model, we use a **specific value** to represent incomplete information: the *null* value so defined:

- Let *null* $\notin$ *dom* be a new symbol.
- We allow the use of *null* in a database instance.
 - Tuples can now use *null* together with values from *dom*.
 - In other words, $t[A]$, for every attribute A , is either a value in *dom* or is *null*.
- We often refer to this model of nulls as **Codd Nulls**.

A Database with Codd Nulls

Some values are now missing

Products

Code	Color
P001	Red
P002	Green
P003	Null

Customers

Code	Name
C001	Eryn
C002	Null
C003	Cora

Orders

Code	Customer	Product
O001	C001	P001
O002	C002	P002
O003	C001	P001

A Database with Codd Nulls – another example

Example: In our domain, county towns have government offices, other cities do not.

We know that:

- **Rome** is a county town; so it has a government office which is in **Via IV Novembre**.
- **Florence** is a county town; so it has a government office, but we **do not know** its address
- **Tivoli** is not a county town; so **it has no** government office
- **Prato** has recently become a county town; has the government office been established?

We don't know

City	GovAddress
Roma	Via IV Novembre
Florence	Null
Tivoli	Null
Prato	Null

Interpretations of the Null Value

We have (at least) three different interpretations of the null value:

- **unknown value**: there is a domain value, but it is **not known** (we do not know the address of the government office in Florence)
- **non-existent value**: the attribute is **not applicable** for the tuple (Tivoli has no government office)
- **no-information value**: we **don't know** whether a value **exists or not** (Prato is a new county town, we do not know if the government already exists or not) - this is the disjunction (logical or) of the other two

The common assumption is that **NULL** represents an **unknown value**.

Handling NULLs

How should we deal with null values?

- *null* = *null*?
- *null* ≠ *null*?
- **Neither!**

Comparisons involving NULLS are evaluated using a three-valued logic that allows for the **unknown** truth value:

- (1 = 1) is **true**.
- (1 = 2) is **false**.
- (1 = NULL) is **unknown**.

Kleene's Three-Valued Logic

Special truth tables are used to handle the third truth value. This is named **Kleene's three-valued logic**.

NOT(A)		AND(A, B)					OR(A, B)				
		$A \wedge B$		B			$A \vee B$		B		
A	$\neg A$			F	U	T			F	U	T
F	T	A	F	F	F	F	A	F	F	U	T
U	U		U	F	U	U		U	U	U	T
T	F		T	F	U	T		T	T	T	T

Where NULL is Used

- The use of NULL was first proposed by Codd in the 70s
 - In the first series of papers presenting the relational model
- SQL uses this notion of NULL!
 - SQL engines evaluate conditions using three truth values
 - Tuples evaluating to true are then returned.
- The following SQL query may not return the whole R. Why?
 - `Select * FROM R WHERE R.a = 1 OR R.a <> 1`

Marked Nulls

Incomplete Databases

- Having a single **NULL** is unsatisfactory in many ways.
 - We cannot say when two unknown values are the **same**
 - The use of three-valued logic is often counterintuitive.
- A more reasoned approach to incomplete information in relational databases is the use of **multiple null values**.
 - We often refer to this model of nulls as **Marked Nulls**.
- More formally, we assume a countably infinite set named **nulls**.
 - We assume the set **nulls** disjoint from the set **dom** of database values.
 - We understand the symbols in **nulls** as *null* values.

A Database with Marked Nulls

Values are missing but we know that **some are the same**.

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Customers

Code	Name
C001	Eryn
C002	Null ₂
C003	Cora

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Evaluating Queries over incomplete databases: Intuition

We call a database instance **complete** if it **does not contain nulls**.

What does it mean to answer a query over a database?

The case of complete databases is clear!

We want to avoid the use of many-valued logics: counterintuitive behavior.

Intuition: in case of missing values, we want to return what is **certain**.

Example

How would you answer the following queries?

- $\{(x, y) \mid Products(x, y)\}$
- $\{(x) \mid \exists y. Products(x, y)\}$
- $\{(x) \mid \exists y. Products(y, x)\}$
- $\{(z) \mid \exists x. \exists y. Products(x, Red) \wedge Orders(y, z, x)\}$

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Models of a Database

To capture the notion of incomplete information, we use the notion of **valuation**.

A valuation for the set **nulls** is a function $v : \mathbf{nulls} \rightarrow \mathbf{dom}$.

- Given a database instance D , we use $v(D)$ to denote the instance obtained by replacing each *null* n occurring in D with $v(n)$
- Clearly, $v(D)$ produces a **complete database**.

The set of **models of a database** D (denote by $\text{Mod}(D)$) is the set:

$$\{v(D) \mid v \text{ is a valuation}\}$$

Models: Example (1\4)

A database with **marked nulls**

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Customers

Code	Name
C001	Eryn
C002	Null ₂
C003	Cora

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Models: Example (2\4)

Different nulls may go into **different** values

Products

Code	Color
a	Red
P002	Green
P003	b

Customers

Code	Name
C001	Eryn
C002	c
C003	Cora

Orders

Code	Customer	Product
O001	C001	a
O002	C002	P002
O003	C001	a

Models: Example (3\4)

Different nulls may go into the **same** value

Products

Code	Color
a	Red
P002	Green
P003	a

Customers

Code	Name
C001	Eryn
C002	a
C003	Cora

Orders

Code	Customer	Product
O001	C001	a
O002	C002	P002
O003	C001	a

Models: Example (4\4)

Nulls may go into values that were **already** in the database

Products

Code	Color
Red	Red
P002	Green
P003	C001

Customers

Code	Name
C001	Eryn
C002	Eryn
C003	Cora

Orders

Code	Customer	Product
O001	C001	Red
O002	C002	P002
O003	C001	Red

Certain Answers

To capture the notion of certainty for query answers, we adopt the notion of **certain answers**.

Consider a FOL query $\mathbf{q} = \{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$

A tuple $\langle c_1, \dots, c_n \rangle \in \mathbf{dom}^n$ is a **certain answer** for $\mathbf{q}$ over a database $\mathbf{D}$, if $\langle c_1, \dots, c_n \rangle \in q(v(\mathbf{D}))$, for **every valuation** v .

$\mathit{cert}(q, \mathbf{D})$ will denote the set of **certain answers** for $\mathbf{q}$ over $\mathbf{D}$.

Exercise 1

What are the **certain answers** for the following queries?

- $\{(x, y) \mid Products(x, y)\}$
- $\{(x) \mid \exists y. Products(x, y)\}$
- $\{(x) \mid \exists y. Products(y, x)\}$
- $\{(z) \mid \exists x. \exists y. Products(x, Red) \wedge Orders(y, z, x)\}$
- $\{(x) \mid \exists y. Products(x, y) \wedge \neg (y = Green)\}$
- $\{(x) \mid \exists y. \exists z. \exists w. Orders(x, y, z) \wedge Products(z, w)\}$

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Exercise 1: Solution (1\6)

What are the **certain answers** for the following queries?

- $\{(x, y) \mid Products(x, y)\}$
 - $\{(P002, Green)\}$

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Exercise 1: Solution (2\6)

What are the **certain answers** for the following queries?

- $\{(x) \mid \exists y. Products(x, y)\}$
 - $\{(P002), (P003)\}$

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Exercise 1: Solution (3\6)

What are the **certain answers** for the following queries?

- $\{(x) \mid \exists y. Products(y, x)\}$
 - {Green, Red}

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Exercise 1: Solution (4\6)

What are the **certain answers** for the following queries?

- $\{(z) \mid \exists x. \exists y. Products(x, Red) \wedge Orders(y, z, x)\}$
 - {C001}

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Exercise 1: Solution (5\6)

What are the **certain answers** for the following queries?

- $\{(x) \mid \exists y. Products(x, y) \wedge \neg (x = Green)\}$
– $\{\}$

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Exercise 1: Solution (6\6)

What are the **certain answers** for the following queries?

- $\{(x) \mid \exists y. \exists z. \exists w. Orders(x, y, z) \wedge Products(z, w)\}$
 - $\{O001, O002, O003\}$

Products

Code	Color
Null ₃	Red
P002	Green
P003	Null ₁

Orders

Code	Customer	Product
O001	C001	Null ₃
O002	C002	P002
O003	C001	Null ₃

Complexity of Certain Answers: Preliminaries

What is the complexity of computing Certain Answers?

We have different kind of complexity:

- **Combined Complexity**: The complexity of the whole problem.
 - **Data Complexity**: The complexity of the problem for a fixed query.
 - **Query Complexity**: The complexity of the problem for a fixed interpretation.
-

Complexity of Certain Answers

Thm: Let q be a FOL query. Checking whether $\bar{a} \in \text{cert}(q, D)$ can be done in EXPTIME in **Combined Complexity** and is NP-Complete in **Data Complexity**.

Conjunctive Query Answering in CWA

Conjunctive Query

- Let $\varphi(x_1, \dots, x_n)$ be a FOL formula with free variables $x_1, \dots, x_n$
- $\varphi(x_1, \dots, x_n)$ is an **Existential Conjunction** if it is of the form

$$\exists y_1, \dots, \exists y_n P_1(\bar{z}_1) \wedge \dots \wedge P_k(\bar{z}_k)$$

- A FOL query $\{(x_1, \dots, x_n) | \varphi(x_1, \dots, x_n)\}$ is a **Conjunctive Query** if $\varphi(x_1, \dots, x_n)$ is an Existential Conjunction.

Union of Conjunctive Queries

- Let $\varphi(x_1, \dots, x_n)$ be a FOL formula with free variables $x_1, \dots, x_n$
- $\varphi(x_1, \dots, x_n)$ is **Union of Existential Conjunctions** if it is of the form

$$\bigvee_i \psi_i(x_1, \dots, x_n)$$

where each ψ_i is an Existential Conjunction with free variables $x_1, \dots, x_n$

- A FOL query $\{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$ is a **Union of Conjunctive Queries** if $\varphi(x_1, \dots, x_n)$ is a Union of Existential Conjunctions.

Naïve Evaluation

Assume an (*incomplete*) database **D** and a query q

The **naïve evaluation** of q over **D** (written $q^\perp(\mathbf{D})$) is defined as follows:

1. We evaluate q over **D** as if **D** was a complete database
2. We discard all the tuples that contain **nulls**

Thm: If q is a UCQ, then $\text{cert}(q, \mathbf{D}) = q^\perp(\mathbf{D})$

Naïve Evaluation – Example

$$q = \{(x, y) \mid Products(x, y)\}$$

D: Products

Code	Color
P1	<i>Null₁</i>
P2	Green
P3	Blue

1. Evaluate the query over the complete database:

$$q(\mathbf{D}) = \{ (P1, \textit{Null}_1), (P2, \textit{Green}), (P3, \textit{Blue}) \}$$

2. Discard tuples with variables:

$$q^\perp(\mathbf{D}) = \{(P2, \textit{Green}), (P3, \textit{Blue})\}$$

Complexity of Certain Answers for UCQs

Thm: Let q be a UCQ. Checking whether $\bar{a} \in \text{cert}(q, \mathbf{D})$ is NP-Complete in **Combined Complexity** and is in LOGSPACE in **Data Complexity**.

Procedure: Compute the **naïve evaluation** $q^\perp(\mathbf{D})$ of q over $\mathbf{D}$

Corollary: Query Answering for UCQs is in PTIME in **Data Complexity**.

Credits

The presentations of this module of the course are based on the original slides of Prof. Giuseppe De Giacomo, Prof. Diego Calvanese, Prof. Marco Console, Prof. Maurizio Lenzerini, and Prof. Domenico Lembo.